ScrollSense - Assignment 2

Sidhartha Durgam | DA24B003 | DA3406 | Modules 2-3

All experimental values below were computed by the supplied drivers. Analytical costs are explicitly estimates. No ML training is required by this assignment.

Reproduction and protocol

Seed DA24B003; SCALE=50; SQLite 3.45.3; Python 3.12.5; Apple M1 (arm64), 8 CPU cores, 8 GiB RAM. Initial free disk approximately 24 GiB. Same SQLite platform as A1; no external dataset/model download. Experiments use one warm-up plus 3 measured execute-and-fetch trials. Report median and all trial values. Index builds, plan extraction, hashing and report creation are excluded from query time. SQLite cache is 64 MiB; temporary storage FILE; automatic indexes ON. OS cache is not flushed, so these are repeated-run timings, not controlled cold-disk measurements. Each new composite index is tested alone on the same database with original A1 indexes retained. Afterward all three coexist for Tasks 3-4. No ANALYZE is run in either before/after condition; plans therefore reflect the engine's default estimates.

The generator retains the A1 schema and data distributions. Its single data-load transaction has foreign_keys ON; WAL is enabled. Necessary fixes are documented in CHANGELOG.md. The fixed categorical pools are not scaled; random-dependent child tables are approximately, rather than exactly, 50x. A1 has one extra AppUser beyond its configured 5,000; the target is the parameter-block volume, not that extra stored row.

Scaled row-count verification

Table	A1 observed	A2 actual	Ratio
AccountStatusPeriod	5,350	267,359	49.974x
AgentSession	2,000	100,000	50.000x
AppUser	5,001	250,000	49.990x
AudioTrack	2,000	100,000	50.000x
Block	250	12,500	50.000x
CreatorPeriod	714	35,714	50.020x
CreatorTierPeriod	1,406	71,430	50.804x
DeclaredInterest	14,903	749,267	50.276x
ExplicitSignal	3,632	180,434	49.679x
Follow	81,450	4,088,957	50.202x
Hashtag	20	20	1.000x
Impression	300,000	15,000,000	50.000x
InferredInterest	74,785	3,748,480	50.123x
InterestCategory	15	15	1.000x
Like	9,493	479,309	50.491x
LikeRetraction	1,218	57,735	47.401x
ModelPricing	6	6	1.000x
ModerationDecision	38,670	1,935,041	50.040x
ModerationState	5	5	1.000x
MonetisationTier	4	4	1.000x
Mute	333	16,666	50.048x
PromptTemplate	11	11	1.000x
Recommendation	16,464	810,605	49.235x
Share	1,818	89,707	49.344x
ToolCall	13,678	675,938	49.418x
Turn	8,995	450,581	50.092x
TurnQualityScore	2,739	134,730	49.189x
UserTurnRating	444	22,542	50.770x
Video	20,000	1,000,000	50.000x
VideoHashtag	29,847	1,499,676	50.245x
ViewSegment	160,059	7,989,952	49.919x
_type_affinity_test	1	1	1.000x

Integrity check: ok. Foreign-key violations: zero. Core A2 counts are exactly 250,000 users, 1,000,000 videos, 15,000,000 impressions and 100,000 agent sessions. Fixed lookup/helper counts remain unchanged by design.

Task 1 - Access paths and cost arithmetic (20 marks)

Selected inherited queries: F1 (audio/video/impression join), F12 (recommendation-to-completion chain) and F11 (session tool-call recursion). F1 and F12 each join multiple named high-volume tables. These are the untuned A2 baselines, before new indexes; A1's original handle/moderation indexes and automatic PK/UNIQUE indexes remain.

1.1 F1: baseline plan

```
10   0 SCAN i
15   0 SCALAR SUBQUERY 1
20  15 SEARCH Impression
33   0 SEARCH v USING INTEGER PRIMARY KEY (rowid=?)
36   0 SEARCH at USING INTEGER PRIMARY KEY (rowid=?)
39   0 USE TEMP B-TREE FOR GROUP BY
83   0 USE TEMP B-TREE FOR count(DISTINCT)
86   0 USE TEMP B-TREE FOR ORDER BY
```

Plan access	Table / source	Classification
SCAN i	Impression	full table scan
SEARCH Impression	Impression	full table scan for unindexed MAX (confirmed by VM Rewind/Next)
SEARCH v USING INTEGER PRIMARY KEY (rowid=?)	Video	rowid / INTEGER PRIMARY KEY lookup
SEARCH at USING INTEGER PRIMARY KEY (rowid=?)	AudioTrack	rowid / INTEGER PRIMARY KEY lookup

Measured median 9.516817 s; trials: 8.749735, 9.516817, 9.548770 s; 10 result rows.

First five actual rows:

```
[50523, 12]
[154, 11]
[97106, 11]
[4992, 10]
[12844, 10]
```

1.3 Planner reasoning: Without a timestamp-leading index, scanning Impression and probing Video/AudioTrack by rowid avoids building a transient index for a one-shot time range; repeatedly searching impressions from every video has no suitable persistent video key and would do more work.

The scalar MAX subquery's bare SEARCH Impression label does not mean an index exists. Its baseline EXPLAIN VM contains Rewind/Next over the table; the opcode inspection was recorded during the original run and can be regenerated by run_experiments.py. Sort/group/distinct temp B-trees are auxiliary structures, not additional base-table access paths.

1.1 F12: baseline plan

```
11   0 SCAN vs
15   0 SEARCH i USING INTEGER PRIMARY KEY (rowid=?)
18   0 SEARCH r USING INTEGER PRIMARY KEY (rowid=?)
21   0 SEARCH t USING INTEGER PRIMARY KEY (rowid=?)
24   0 SEARCH s USING INTEGER PRIMARY KEY (rowid=?)
27   0 USE TEMP B-TREE FOR GROUP BY
77   0 USE TEMP B-TREE FOR count(DISTINCT)
80   0 USE TEMP B-TREE FOR ORDER BY
```

Plan access	Table / source	Classification
SCAN vs	ViewSegment	full table scan
SEARCH i USING INTEGER PRIMARY KEY (rowid=?)	Impression	rowid / INTEGER PRIMARY KEY lookup
SEARCH r USING INTEGER PRIMARY KEY (rowid=?)	Recommendation	rowid / INTEGER PRIMARY KEY lookup
SEARCH t USING INTEGER PRIMARY KEY (rowid=?)	Turn	rowid / INTEGER PRIMARY KEY lookup
SEARCH s USING INTEGER PRIMARY KEY (rowid=?)	AgentSession	rowid / INTEGER PRIMARY KEY lookup

Measured median 0.350886 s; trials: 0.479358, 0.350886, 0.336636 s; 3,363 result rows.

First five actual rows:

```
[3, 12, 391141, 3, 1]
[37, 156, 649605, 6, 1]
[150, 644, 716311, 2, 1]
[151, 651, 820989, 2, 1]
[256, 1100, 443464, 5, 1]
```

1.3 Planner reasoning: The completed=1 predicate has no leading persistent index at baseline, so scanning ViewSegment then probing Impression and the recommendation/turn/session chain by rowid avoids repeated unindexed completion searches from every recommendation.

1.1 F11: baseline plan

```
  2   0 CO-ROUTINE CallTree
  5   2 SETUP
  8   5 SCAN tc
 12   5 SEARCH t USING INTEGER PRIMARY KEY (rowid=?)
 43   2 RECURSIVE STEP
 45  43 SCAN ct
 48  43 BLOOM FILTER ON tc (parent_call_id=?)
 61  43 SEARCH tc USING AUTOMATIC COVERING INDEX (parent_call_id=?)
 88   0 SCAN CallTree
102   0 USE TEMP B-TREE FOR ORDER BY
```

Plan access	Table / source	Classification
SCAN tc	ToolCall	full table scan
SEARCH t USING INTEGER PRIMARY KEY (rowid=?)	Turn	rowid / INTEGER PRIMARY KEY lookup
SCAN ct	ct	CTE / recursive intermediate scan
SEARCH tc USING AUTOMATIC COVERING INDEX (parent_call_id=?)	ToolCall	automatic transient index lookup (constructed during execution)
SCAN CallTree	CallTree	CTE / recursive intermediate scan

Measured median 0.452884 s; trials: 0.452884, 0.451461, 0.453942 s; 6 result rows.

First five actual rows:

```
[1, 1, None, 1, 'search_hashtags', 0, 608, '1']
[2, 2, 1, 1, 'search_videos', 0, 454, '1.2']
[1, 3, None, 3, 'fetch_trending_audio', 0, 775, '3']
[1, 4, None, 3, 'fetch_trending_audio', 0, 239, '4']
[2, 5, 4, 3, 'search_hashtags', 0, 93, '4.5']
```

1.3 Planner reasoning: The root setup scans ToolCall and probes Turn by PK because no turn-leading ToolCall index supports the inverse session-first access; repeatedly scanning ToolCall for each session turn would do more work, while recursion constructs an automatic parent index instead of a fresh scan for each parent.

1.2 One full-table-scan calculation: F1 Impression only

Use the baseline SCAN i step, N=15,000,000 rows. Assume B=4,096 bytes/block. Use a deliberately conservative fixed-width budget L=96 bytes/row: five integer slots x 8=40 bytes, model_version rank-v1 style text=7 bytes, ISO timestamp=20 bytes, plus 29 bytes reserved for record header/null/slot/allocation overhead; 40+7+20+29=96. Charge the optional recommendation slot even when NULL. SQLite actually uses compact varints and variable-length records, so this is an explicit model rather than a measured row width. Rows/block=floor(B/L)=floor(4096/96)=42. Blocks transferred=ceil(N/42)=ceil(15,000,000/42)=357,143. Modeled bytes=357,143 x 4,096=1,462,857,728. This estimates one uncached sequential pass, excluding index pages, overflow, internal B-tree pages and OS/SQLite cache hits. SQLite's actual page_size is 4,096 bytes. Equal assumed/actual block size does not make the transfer estimate measured physical I/O.

Task 2 - Composite index design (25 marks)

Every tested index uses two columns of the SAME table that appear in the original query's predicates/joins. A table's INTEGER PRIMARY KEY may also be implicitly present in secondary indexes; the declared column order below states the logical access design.

2.3 Rejected before building

Rejected Impression(video_id,impressed_at) for F1. The query constrains time globally and does not provide a specific video_id; a timestamp suffix cannot provide a normal leftmost-prefix range search. A skip-scan can sometimes help when statistics and leading-key duplication justify it, but is not equivalent to the desired timestamp-leading range. It would also cost an extra 15-million-row index to test a design without a better justification. This rejected candidate was never built.

2.1 / 2.2 Actual isolated comparisons

Query	Before median s	After median s	Before/after	Index build s	Rows equal
F1	9.516817	0.530438	17.941x	14.050	SHA256 match
F12	0.350886	0.077243	4.543x	4.270	SHA256 match
F11	0.452884	0.043517	10.407x	0.517	SHA256 match

F1: column order and measured plan

```
-- Apply only AFTER recording the baseline. SQLite 3.44+.
-- F1: timestamp range and MAX first, video join payload second (covering).
CREATE INDEX IF NOT EXISTS a2_impression_time_video
    ON Impression(impressed_at, video_id);
```

Impression.impressed_at serves the range WHERE and scalar MAX; video_id serves the join to Video and makes this access covering. Leftmost-prefix: timestamp-only filtering can seek the first key. Reversing to (video_id,impressed_at) leaves the timestamp filter without a fixed leading video_id; it cannot provide the same direct global range seek. Once a range is used on the first column, the second generally covers rather than tightens a global range.

Before plan:

```
 10   0 SCAN i
 15   0 SCALAR SUBQUERY 1
 20  15 SEARCH Impression
 33   0 SEARCH v USING INTEGER PRIMARY KEY (rowid=?)
 36   0 SEARCH at USING INTEGER PRIMARY KEY (rowid=?)
 39   0 USE TEMP B-TREE FOR GROUP BY
 83   0 USE TEMP B-TREE FOR count(DISTINCT)
 86   0 USE TEMP B-TREE FOR ORDER BY
```

After plan:

```
 10   0 SEARCH i USING COVERING INDEX a2_impression_time_video (impressed_at>?)
 13   0 SCALAR SUBQUERY 1
 18  13 SEARCH Impression USING COVERING INDEX a2_impression_time_video
 34   0 SEARCH v USING INTEGER PRIMARY KEY (rowid=?)
 37   0 SEARCH at USING INTEGER PRIMARY KEY (rowid=?)
 40   0 USE TEMP B-TREE FOR GROUP BY
 84   0 USE TEMP B-TREE FOR count(DISTINCT)
 87   0 USE TEMP B-TREE FOR ORDER BY
```

Before trials s: 8.749735, 9.516817, 9.548770. After trials s: 0.530438, 0.529566, 0.537518.

The plan names the new index and median improves by 17.941x in this repeated-run protocol. Remaining row retrieval and result sorting still contribute; this is not a universal production guarantee.

F12: column order and measured plan

```
-- F12: selective completed=1 equality first, impression join key second.
-- Covering for all ViewSegment columns read by F12.
CREATE INDEX IF NOT EXISTS a2_segment_completed_impression
    ON ViewSegment(completed, impression_id);
```

ViewSegment.completed serves WHERE/ON completed=1; impression_id serves JOIN Impression and the completion grouping/count chain. Equality on completed allows a narrow index range; impression_id then orders/covers those matches. Reversing (impression_id,completed) can find completions for a known impression but leaves the global completed predicate without a fixed first key. The original PK starts with impression_id and already supplies that alternative direction.

Before plan:

```
 11   0 SCAN vs
 15   0 SEARCH i USING INTEGER PRIMARY KEY (rowid=?)
 18   0 SEARCH r USING INTEGER PRIMARY KEY (rowid=?)
 21   0 SEARCH t USING INTEGER PRIMARY KEY (rowid=?)
 24   0 SEARCH s USING INTEGER PRIMARY KEY (rowid=?)
 27   0 USE TEMP B-TREE FOR GROUP BY
 77   0 USE TEMP B-TREE FOR count(DISTINCT)
 80   0 USE TEMP B-TREE FOR ORDER BY
```

After plan:

```
 11   0 SEARCH vs USING COVERING INDEX a2_segment_completed_impression (completed=?)
 15   0 SEARCH i USING INTEGER PRIMARY KEY (rowid=?)
 18   0 SEARCH r USING INTEGER PRIMARY KEY (rowid=?)
 21   0 SEARCH t USING INTEGER PRIMARY KEY (rowid=?)
 24   0 SEARCH s USING INTEGER PRIMARY KEY (rowid=?)
 27   0 USE TEMP B-TREE FOR GROUP BY
 77   0 USE TEMP B-TREE FOR count(DISTINCT)
 80   0 USE TEMP B-TREE FOR ORDER BY
```

Before trials s: 0.479358, 0.350886, 0.336636. After trials s: 0.080180, 0.077182, 0.077243.

The plan names the new index and median improves by 4.543x in this repeated-run protocol. Remaining row retrieval and result sorting still contribute; this is not a universal production guarantee.

F11: column order and measured plan

```
-- F11: parent root/recursive equality first; turn join second. Both keys are
-- bound in root setup, while parent-only recursive probes retain the prefix.
CREATE INDEX IF NOT EXISTS a2_toolcall_parent_turn
    ON ToolCall(parent_call_id, turn_id);
```

ToolCall.parent_call_id serves root IS NULL and recursive parent equality; turn_id serves the join to selected Turn rows. Parent-first retains a useful prefix for EVERY recursive child probe; root setup can bind parent NULL plus turn_id, using both keys. Reversing to (turn_id,parent_call_id) still uses both equality keys in root setup but leaves parent-only recursion without a fixed leading turn_id, requiring a scan or transient parent index. There is no invented claim that order matters for two already-bound equalities: the decisive distinction is the parent-only recursive access. A turn-only query would favor the reverse order; this design is specific to F11.

Before plan:

```
  2   0 CO-ROUTINE CallTree
  5   2 SETUP
  8   5 SCAN tc
 12   5 SEARCH t USING INTEGER PRIMARY KEY (rowid=?)
 43   2 RECURSIVE STEP
 45  43 SCAN ct
 48  43 BLOOM FILTER ON tc (parent_call_id=?)
 61  43 SEARCH tc USING AUTOMATIC COVERING INDEX (parent_call_id=?)
 88   0 SCAN CallTree
102   0 USE TEMP B-TREE FOR ORDER BY
```

After plan:

```
  2   0 CO-ROUTINE CallTree
  5   2 SETUP
  9   5 SEARCH tc USING INDEX a2_toolcall_parent_turn (parent_call_id=?)
 15   5 SEARCH t USING INTEGER PRIMARY KEY (rowid=?)
 46   2 RECURSIVE STEP
 49  46 SCAN ct
 50  46 SEARCH tc USING INDEX a2_toolcall_parent_turn (parent_call_id=?)
 77   0 SCAN CallTree
 91   0 USE TEMP B-TREE FOR ORDER BY
```

Before trials s: 0.452884, 0.451461, 0.453942. After trials s: 0.043517, 0.043622, 0.043278.

The plan names the new index and median improves by 10.407x in this repeated-run protocol. Remaining row retrieval and result sorting still contribute; this is not a universal production guarantee.

The measured after-plan binds ONLY parent_call_id in root setup and recursion; it then probes Turn by rowid. It does not bind turn_id as a second search key. The second column supplies the join value while parent-only recursion gains direct

indexed access and the former automatic recursive index disappears. Scanning the common root prefix remains work, explaining why the measured gain is finite. A session-first order could bind both keys, but that is not the order chosen in these un-ANALYZEd plans.

Index storage/write tradeoff: all three additions consume disk and make future inserts maintain additional B-trees. Build time is reported separately; original generator does not build tuning indexes during its load.

Task 3 - Join algorithms (20 marks)

3.1 Two joins of large tables

J1 literal SQL:

```
SELECT count(*) AS segments, sum(vs.segment_end_ms-vs.segment_start_ms) AS watch_ms
FROM ViewSegment vs JOIN Impression i ON i.impression_id=vs.impression_id;
```

```
  4   0 SCAN vs
  6   0 SEARCH i USING INTEGER PRIMARY KEY (rowid=?)
```

SQLite uses nested scans: the outer SCAN loop drives the inner SEARCH lookup. Both pairs contain large event tables, not lookup dimensions. The EXPLAIN VM inspection recorded during the original run shows Rewind/Next and SeekRowid instructions, supporting an index nested-loop classification rather than a hash or sort-merge join.

Actual median 1.387817 s; result [[7989952, 115607461624]].

J2 literal SQL:

```
SELECT count(*) AS calls, sum(tc.latency_ms) AS latency_ms
FROM ToolCall tc JOIN Turn t ON t.turn_id=tc.turn_id;
```

```
  4   0 SCAN tc
  6   0 SEARCH t USING INTEGER PRIMARY KEY (rowid=?)
```

SQLite uses nested scans: the outer SCAN loop drives the inner SEARCH lookup. Both pairs contain large event tables, not lookup dimensions. The EXPLAIN VM inspection recorded during the original run shows Rewind/Next and SeekRowid instructions, supporting an index nested-loop classification rather than a hash or sort-merge join.

Actual median 0.093186 s; result [[675938, 277243656]].

3.2 Analytical costs for J1: ViewSegment joined to Impression

Input tuples: nR=7,989,952; nS=15,000,000. Observed TABLE leaf pages: BR=35,166, BS=180,238. The SQLite build lacks dbstat, so the driver traverses documented rowid-table B-tree page headers after checkpoint and verifies that leaf-cell totals equal SQL COUNT(*). These are actual storage cardinalities, not measured transfers. Model M=4,096 buffer pages (16 MiB at 4 KiB/page), adequate output streaming, no ordering requirement and uniform partitioning. Estimates omit overflow/internal/index-page transfer, writing the aggregate output and cache reuse. For clarity we distinguish block nested loops from the actual indexed loop.

Method	Formula and modeled page I/O	Estimate
Block nested loop	BR + ceil(BR/(M-2)) x BS	1,657,308
External sort-merge	sort(BR)+sort(BS)+BR+BS	1,077,020
Grace hash (two-pass)	3 x (BR+BS)	646,212
Indexed nested loop, cold upper model	BR + nR x h, assume h=3	24,005,022

sort(B)=2B[1+ceil(log_(M-1)(ceil(B/M)))], including initial runs and merge passes, followed by one scan of both sorted inputs for the join. Hash feasibility assumes BR <= (M-1)(M-2), so each uniformly distributed build partition fits the second-pass buffers; skew can break this assumption. An in-memory hash would instead cost approximately BR+BS if the full build side fits, which is not assumed here.

A simple tuple nested-loop baseline would examine nR x nS = 119,849,280,000,000 candidate pairs; that CPU comparison count is separate from the page-I/O table. Under this specific external-memory model, Grace hash has the smallest listed I/O cost. The actual SQLite strategy remains indexed nested loops: SQLite does not implement native hash or sort-merge joins. Its cached upper B-tree levels and repeated page reuse also make the cold h-per-tuple bound pessimistic. Analytical cheapest and supported engine choice do not agree here; no estimated I/O is presented as measured runtime.

3.3 Three-table join-order challenge

```
SELECT count(*) AS completed_segments
FROM Video v JOIN Impression i ON i.video_id=v.video_id
JOIN ViewSegment vs ON vs.impression_id=i.impression_id
WHERE v.owner_id=154384 AND vs.completed=1;
```

Estimate BEFORE consulting the plan (recorded during the original run): choose the owner of the first video, independent of optimizer output, guaranteeing a nonempty base selection. Predicate owner_id=154384 selects 23/1,000,000 videos (measured base selectivity); completed=1 selects 42,604/7,989,952 segments. With a uniform distribution and independent predicates, a Video-first order estimates 23 -> 345.0 matching impressions -> 0.98 completed segments; a completion-first order estimates 42,604 -> 42,604 segment/impression matches -> 0.98 after the owner filter. ViewSegment is not deduplicated here; every segment retains its parent, so the middle completion-first cardinality equals selected segments.

My initial logical heuristic favors filtered Video -> Impression -> filtered ViewSegment because of smaller estimated intermediate results. But cardinalities alone are insufficient to certify physical cheapest: no owner_id-leading Video index or video_id-leading Impression index was added. Video-first requires scanning the 15-million-row Impression relation (or building a large transient video index). The completion-first order can range-search the new completed-leading index and probe Impression and Video by PK, using far fewer rows than a full Impression scan. This access-path cost can overturn the smaller-intermediate heuristic. Correlation/popularity skew can also invalidate uniform estimates.

Actual plan:

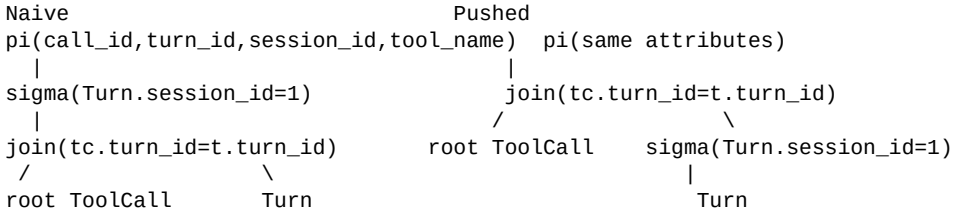
```
  5   0 SEARCH vs USING COVERING INDEX a2_segment_completed_impression (completed=?)
  9   0 SEARCH i USING INTEGER PRIMARY KEY (rowid=?)
 12   0 SEARCH v USING INTEGER PRIMARY KEY (rowid=?)
```

Observed final completed-segment count: 1; median 0.117193 s. The engine actually starts from the completed-leading ViewSegment index, then probes Impression and Video by PK. It disagrees with my initial smallest-intermediate heuristic, and its order is justified for these available access paths: 42,604 qualifying segment probes avoid a 15-million-row Impression scan. The predicted final count 0.98 is close to the observed 1, but that alone does not validate uniformity generally. This reconciles the disagreement without claiming the plan proves global optimality.

Task 4 - Predicate pushdown (15 marks)

4.1 Algebra trees and equivalence

This uses root ToolCall-Turn rows with selective session_id=1, different from A1 C3's live-moderation/watch rewrite. Both variants first restrict ToolCall to parent_call_id IS NULL; that shared root condition is unchanged. Define R=selected root ToolCall and S=Turn. Qualify attributes to avoid ambiguity. Projection keeps call_id,turn_id,session_id,tool_name; order is call_id.



Rule: sigma_p(R join_theta S) = R join_theta sigma_p(S) when p references only S attributes and the join is inner. SQL bag multiplicities are preserved; both SQLs retain exactly the join/projection attributes and final order. Counterexample: for R LEFT JOIN S, a WHERE S.flag=1 removes NULL-extended unmatched R rows; pushing it into S while retaining the LEFT JOIN preserves those rows. That rewrite is not equivalent without also changing the outer join to an inner join or proving a suitable predicate condition.

4.2 Literal SQL and actual measurements

P_naive

```
SELECT joined.call_id, joined.turn_id, joined.session_id, joined.tool_name
FROM (SELECT tc.call_id,tc.turn_id,t.session_id,tc.tool_name
      FROM ToolCall tc JOIN Turn t ON t.turn_id=tc.turn_id
      WHERE tc.parent_call_id IS NULL) joined
WHERE joined.session_id=1 ORDER BY joined.call_id;
```

```
  5   0 SEARCH tc USING INDEX a2_toolcall_parent_turn (parent_call_id=?)
 11   0 SEARCH t USING INTEGER PRIMARY KEY (rowid=?)
 23   0 USE TEMP B-TREE FOR ORDER BY
```

Median 0.043775 s; trials 0.043738, 0.043812, 0.043775 s; result rows=4; SHA256=2e4d52a8337c86fb77efa25779c6ad03cb79b9eb7ff7147ef8c5d638a4a00ca0.

P_pushed

```
SELECT tc.call_id,tc.turn_id,t.session_id,tc.tool_name
FROM ToolCall tc JOIN (SELECT turn_id,session_id FROM Turn WHERE session_id=1) t
ON t.turn_id=tc.turn_id WHERE tc.parent_call_id IS NULL ORDER BY tc.call_id;
```

```
  5   0 SEARCH tc USING INDEX a2_toolcall_parent_turn (parent_call_id=?)
 11   0 SEARCH Turn USING INTEGER PRIMARY KEY (rowid=?)
 23   0 USE TEMP B-TREE FOR ORDER BY
```

Median 0.043762 s; trials 0.043762, 0.043718, 0.044047 s; result rows=4; SHA256=2e4d52a8337c86fb77efa25779c6ad03cb79b9eb7ff7147ef8c5d638a4a00ca0.

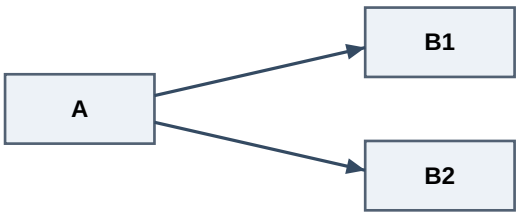
Naive-minus-pushed median difference=0.000013 s. The physical plans are identical after normalizing alias t to table Turn; the raw aliases are preserved above. Equal result hashes prove output equivalence on this dataset. Both plans search root ToolCall using the same parent-leading index, probe Turn by PK, and use the same sort. Neither contains MATERIALIZE for the derived full join or filtered Turn subquery. EXPLAIN VM inspection during the original run shows the session_id comparison (Column then Ne) within the lookup loop BEFORE SorterInsert/ResultRow. Thus the optimizer has flattened both SQL forms and applies the same early per-row predicate; spelling the logical pushdown separately does not reduce the executed work. The approximately 13-microsecond median gap is smaller than the measured trial spread and supports no meaningful speedup claim. The optimizer did not choose a session-first range access; that is a join-order choice, not proof that the literal naive syntax materialized an unfiltered join.

Task 5 - Schedules, anomalies and isolation (20 marks)

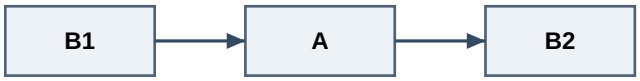
5.1 A1 moderation schedule and precedence graph

Observed: A begins and appends an uncommitted decision; B reads live; A commits; B's next read returns taken_down. A1's B reads were autocommit SELECTs, hence separate database transactions B1 and B2. In WAL, the first reader selects the earlier committed version. Name the shared logical data item current moderation state of video 1; the insert changes that derived value.

Abstract conflict graph (acyclic)



WAL committed-version order




```
Observed operation order: wA(D); rB1(D_old); commitA; rB2(D_new)
Conflict edges: A -> B1 and A -> B2. No reverse edges; acyclic.
Conflict-equivalent serial order: A, B1, B2 (B1/B2 interchangeable).
WAL committed-version dependencies: B1 -> A -> B2.
Value-preserving snapshot serial order: B1, A, B2.
```

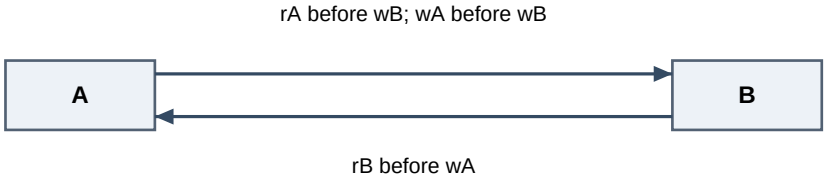
Under the conventional conflict-graph test applied to this operation order and one abstract moderation item D, the schedule IS conflict-serializable: all write/read conflict edges leave A, so the graph is acyclic; a topological serial order is A,B1,B2. There is an important distinction from value-preserving serialization: B1 reads an OLD committed version while A's new version is uncommitted. WAL is multiversion, so B1,A,B2 is the committed-version order reproducing those observed values; the simple single-version graph does not capture this version visibility. On physical append-only rows, old and new decisions are distinct version items. If B instead held ONE explicit snapshot transaction across both SELECTs, it would continue to read live, with snapshot serialization B before A. Treating B1/B2 as one database transaction while showing different committed values would incorrectly model the actual transaction boundaries.

Actual two-connection evidence: before=live, after=taken_down. The same driver also verifies real retraction failure rollback and rejects a third handle change under an atomic guard.

5.2 Own-schema nonserializable logical operation schedule

Two profile editors append |A and |B to AppUser(user_id=1).display_name. Each unsafe application operation reads in autocommit and later submits an absolute update. Both read base. A writes base|A; B later writes base|B, overwriting A's edit. The operations are logical read/modify/write transactions intended by the application, but are incorrectly split into multiple actual SQLite transactions. This is an application lost update, not a claim that SQLite permits a lost update inside proper concurrent WAL snapshot transactions.

Lost update: cycle between logical profile edits




```
rA(x='base'); rB(x='base'); wA(x='base|A'); wB(x='base|B')
A ---> B: rA before wB, and wA before wB
B ---> A: rB before wA
Cycle A -> B -> A: NOT conflict-serializable as logical edits.
```

A plain request to run two deferred read/write SQLite transactions would instead often raise SQLITE_BUSY_SNAPSHOT; the regression test confirms that. The deliberately unsafe operation boundaries above create the requested real bad outcome on the actual ScrollSense AppUser table without disabling isolation or inventing another database's behavior.

5.3 Real execution and prevention

The threads open independent SQLite connections. Events coordinate A's read, B's old read/lock attempt, A's commit, and B's update/retry, with logged perf_counter timestamps; no sleeps are used to guess order. Unsafe final value: base|B. Protected final value: base|A|B. Protected B lock failure: SQLITE_BUSY (5), 'database is locked'. Assertions require these outcomes.

Both protected operations execute BEGIN IMMEDIATE BEFORE reading, reserve the sole writer, read a fresh value, update and COMMIT. B's immediate attempt at the same scheduled point fails with timeout=0; after A commits B retries the entire operation and reads base|A. Graph of completed protected edits is A->B, acyclic.

Cost sentence 1: BEGIN IMMEDIATE reserves SQLite's database-wide writer, so B waits with a busy timeout or receives SQLITE_BUSY while A's operation is open. Cost sentence 2: B must retry the whole read/modify/write operation and re-read the latest value, adding latency while unrelated WAL readers can continue.

Actual timestamped schedule (seconds relative to driver start):

Seconds	Actor	Action	Actual value
0.009729	A	BEGIN; append uncommitted moderation decision	taken_down
0.010146	B1	autocommit read while A write is uncommitted	live
0.010238	A	COMMIT moderation decision	None
0.010252	B2	new autocommit read after A commit	taken_down
0.010370	T1	duplicate retraction failure and ROLLBACK	UNIQUE constraint failed: LikeRetraction.like_id
0.010877	T3	third handle change rejected atomically	2
0.014411	bad A	read display_name	base
0.014772	bad B	read same old display_name	base
0.014949	bad A	write and commit	base / A
0.014957	bad B	read value used for update	base

Seconds	Actor	Action	Actual value
0.015152	bad B	write and commit	base / B
0.017816	fixed A	BEGIN IMMEDIATE	None
0.018097	fixed A	read display_name	base
0.018180	fixed B	BEGIN IMMEDIATE denied while A owns lock	{'message': 'database is locked', 'code': 5, 'name': 'SQLITE_BUSY'}
0.018344	fixed A	write and commit	base / A
0.018357	fixed B	retry entire operation; BEGIN IMMEDIATE acquired	None
0.018798	fixed B	read value used for update	base / A
0.018880	fixed B	write and commit	base / A / B

LLM-use appendix

I used an LLM assistant to help review the A1 code, suggest indexes, write the experiment and concurrency scripts, and draft parts of the report. The reported timings, row counts, and outcomes came from running the code.

One mistake was splitting SQL statements at every semicolon. This broke F5 because it contains `';'` inside a quoted string. Comparing the parsed query with the original exposed the problem. The parser was changed to use `sqlite3.complete_statement()`, and a regression test was added to check this case.

I am responsible for understanding and explaining the submitted work, including the parts developed with LLM assistance.

Limitations and final validation

Scale50 is actually loaded and experiments actually ran; no partial dataset is substituted. Fixed seed reproduces data under this Python version, but timings vary with cache, storage, competing processes and SQLite planner versions. Three repeated timings support a descriptive comparison, not significance testing. Input leaf-page cardinalities come from read-only B-tree traversal; cost formulas remain simplified estimates. Pre-existing chronology/other A1 analytics defects are disclosed in the changelog and not silently redesigned. No production scalability or ML quality claim follows from these synthetic measurements.

Verification: integrity and FK audits, exact scale counts, index-result and pushdown-result SHA256 checks, deterministic sequence regression, native WAL stale-upgrade regression, live two-connection bad/fixed schedule assertions, portable paths and source-preservation audit. REQUIREMENTS.md maps each assignment requirement to the report and source files. The submission contains one PDF and the source repository. Original measurements are presented here and in plan_analysis.md; raw JSON files and logs are retained locally. The results/ folder is not submitted and is created by rerunning the README commands. No pre-existing output files are needed to run the submitted scripts.